시스템 프로그래밍

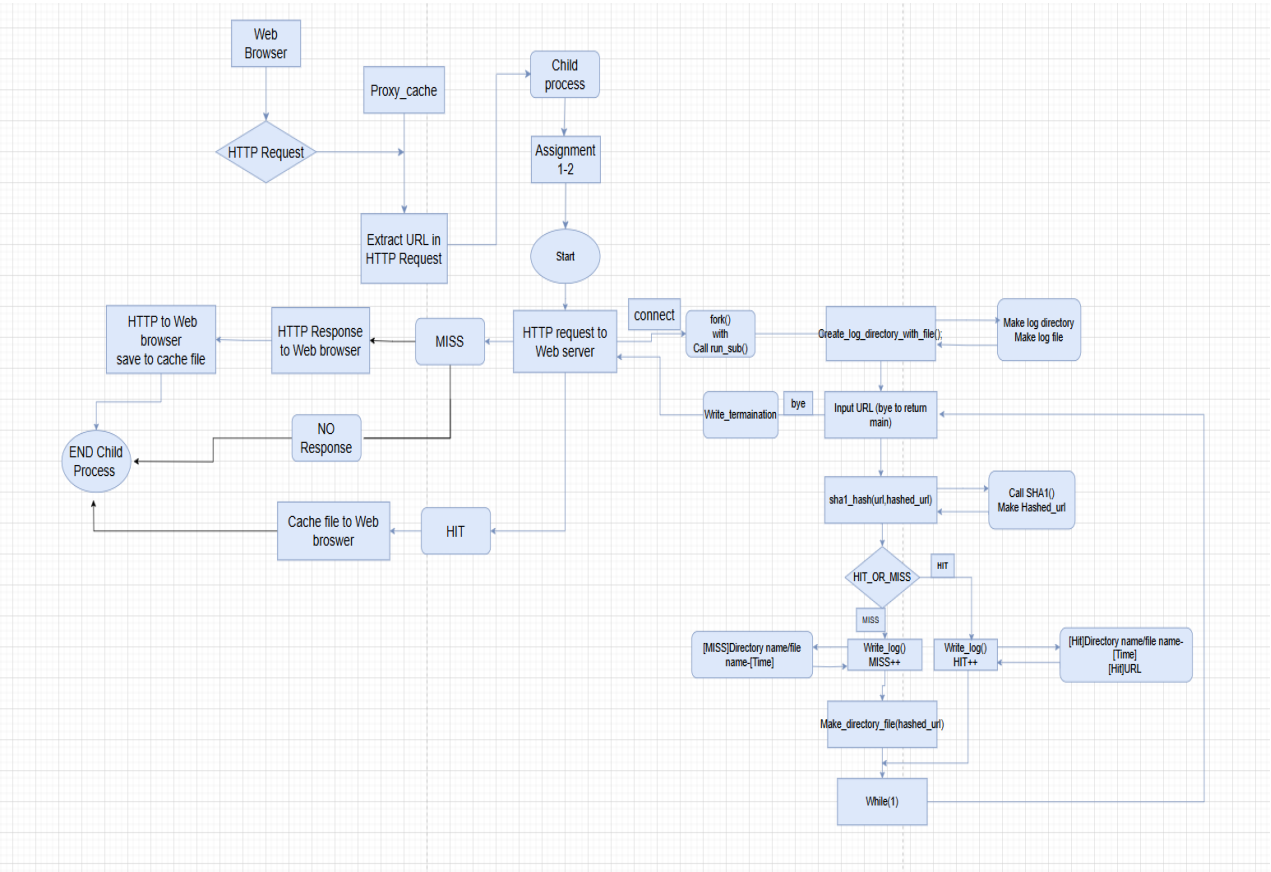
Proxy 2-4

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

Introduction

이번 과제는 이전 proxy 2-3 과제에 이어서, 마찬가지로, 웹 브라우저로부터 연결을 받고, 이후 자식 프로세서에서 HTTP request를 받은 뒤에, request header로부터 웹 서버의 URL을 추출하여, URL 해시 후 각각에 대해서 Cache를 생성하고, 이 Cache를 바탕으로, 입력되는 URL 이 HIT인지 MISS인지를 판별하여, logfile에 작성합니다. 또한 SIGINT 신호는 "Ctrl c" 를 입력받으면 , 신호가 발생하고, 이를 처리하여 proxy 서버가 종료되게 하여서, 전체 실행시간과 총 처리한 자식 프로세스 수 또한 logfile에 작성합니다. Signal 을 다루는 방법은 3가지로 default , Ignore, catch 가 있는데, 이중에서 catch 방법이고, 추가적인 Signal handler function을 통하여, 이를 감지할 것입니다.

Flow chart



Pseudo code

```
    SET Register SIGALRM handler
    SET Register SIGCHLD handler
    SET Register SIGINT handler
    Create, bind, and listen on a socket
While true:
    Accept client connection
    Fork a new child process
     In child process:
          Read HTTP request from browser
          Parse method and URL
          Hash the URL (SHA1)
          Check cache (HIT or MISS)
      If HIT
               Log if not already logged
                Read cached file and send to browser
      If MISS
                Create cache directories
                Extract host and path from URL
                Set alarm for timeout
                Connect to web server
                Send GET request
                Receive response, send to browser, and save to cache
                Close connection and exit child
     In parent process:
     Close client socket and loop
     Close main server socket
```

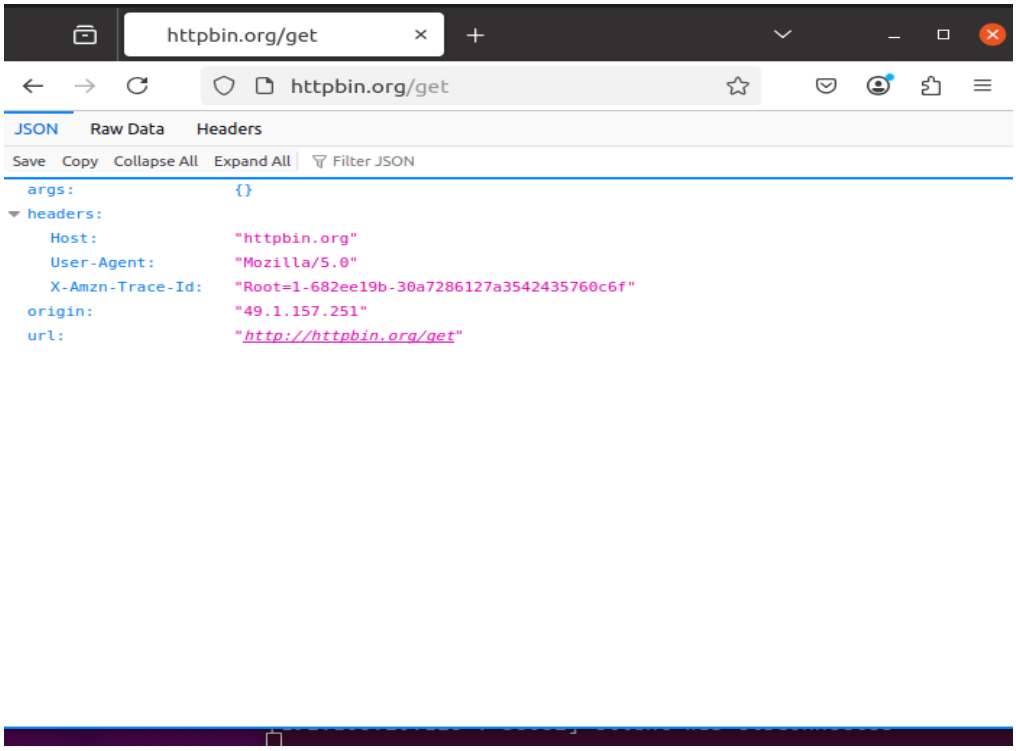
결과화면

```
/home/kw2021202089/cache
├── 06b
│   └── 07eeaf733d9447b68feda5fd8639c9069167c
├── 073
│   └── 05ea2c60102e6adfa48c6e0e5f76eba1a9c48
├── 344
│   └── 6ed10558b1d7286acf8b24681c6ef3abdb3e2
├── 41c
│   └── 39e388e2702c32b3bff39527b39c6eebc6207
├── 41e
│   └── 86c40458831e465828f3cf87979f02fb5defc
├── 446
│   └── 7b1a4bc29b49846dd1ed35d8d9c8d7c23df8d
├── 44a
│   └── 9a68c0ca86342fead55c8dd129f0f183fa96a
├── 479
│   └── 76947b24fd2f6a2bcddc75cf6c14c41a1d6ab
├── 565
│   └── 73643331b10dd1a32919b722a5982e85e6662
├── 568
│   └── e5084d36f5b00a964d08585a86403a978c54a
├── 743
│   └── f144edc031334b36b6aa9e5c20fb46ccdcfbb
├── 78d
│   └── 53be50cba5e6aba037435b0c677e8187ef976
├── 80f
│   └── c9939d8ed544a0a87653614f55dd18e589342
├── 88e
│   └── 5214184841ac3411dbb6246cd8f70c5a7e19a
├── 90a
│   └── 9613720321da1feceb63d4008769be55903fb
├── b4a
│   └── 4ccd9f36356d6a4166ffcdf6830f7118500a8
├── c20
│   └── ff7b5cbc0cb6ed6d2543fde4d92ecf3b8d5c7
├── d33
│   └── 9a3ee5e6b4b0d3255bfef95601890afd80709
├── e11
│   └── 2bfc32a27f0b118e28ff366548db647975397
├── e64
│   └── a0a6639cc92cb8f1601e0260c54fb2b0c3a5e
└── e73
    └── 10497f395413192ef6b26fc044411448cd038

21 directories, 21 files
```

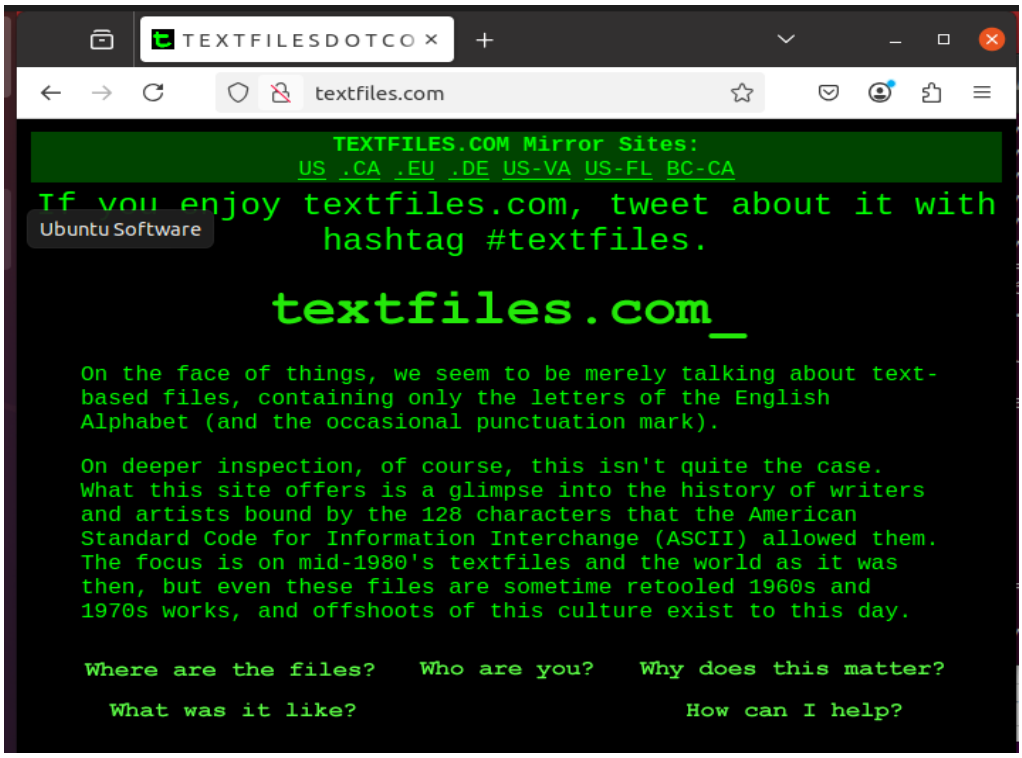
⇨ Cache 디렉토리들이 형성된 것을 확인할 수 있습니다.

```
HTTP/1.1 200 OK
date: Thu, 22 May 2025 12:48:33 GMT
server: Apache
last-modified: Fri, 22 Oct 2010 20:34:17 GMT
accept-ranges: bytes
content-length: 2787
vary: Accept-Encoding
content-type: text/html; charset=utf-8
connection: close

<?xml version="1.0" encoding="iso-8859-1"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<link rev="made" href="mailto:esr@snark.thyrsus.com" />
<link rel="stylesheet" href="jargon.css" type="text/css"/>
<meta name="description" content="Indispensable dictionary of hacker slang" />
<title>Jargon File Resources</title>
</head>
<body>
<table width='100%' cellpadding='0' summary='Canned page header'>
<tr>
<td width='30%'>Back to <a href='http://catb.org/~esr/'>Eric's Home Page</a></td>
<td width='30%' align='center'>Up to <a href='/~esr/sitemap.html'>Site Map</a></td>
<td width='30%' align='right'>01 Oct 2004</td>
</table>
<hr />

<img src='graphics/glider-small.png' align='left'/>
<h1 class="centered">The Jargon File, version 4.4.8</h1>

<p>This page indexes all the WWW resources associated with the Jargon File
and its print version, <cite>The New Hacker's Dictionary</cite>. It's as
official as anything associated with the Jargon File gets.</p>

<p>On 23 October 2003, the Jargon File achieved the
dubious honor of being cited in the SCO-vs.-IBM lawsuit.  See the <a
href='html/F/FUD.html'>FUD</a> entry for details.</p>


<ul>
<li><a href="html/index.html">Browse the Jargon File.</a></li>

<li><a href="news.html">What's new in the Jargon File.</a></li>

```

⇨ Cache 디렉토리 중 아무 캐시 파일을 확인 했을 때, response data가 작성도니 것을 확인할 수 있습니다.

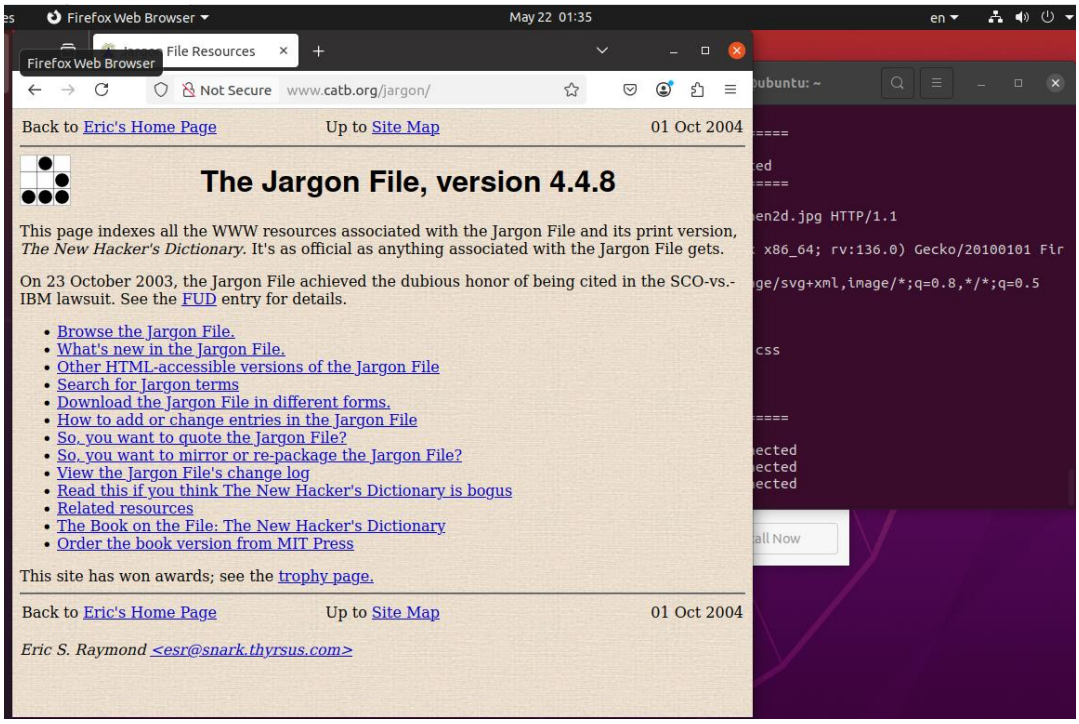


⇨ http://httpbin.org/get를 입력했을 때, 프록시 서버로부터 받은 response message가 정상적으로 출력된 화면입니다.

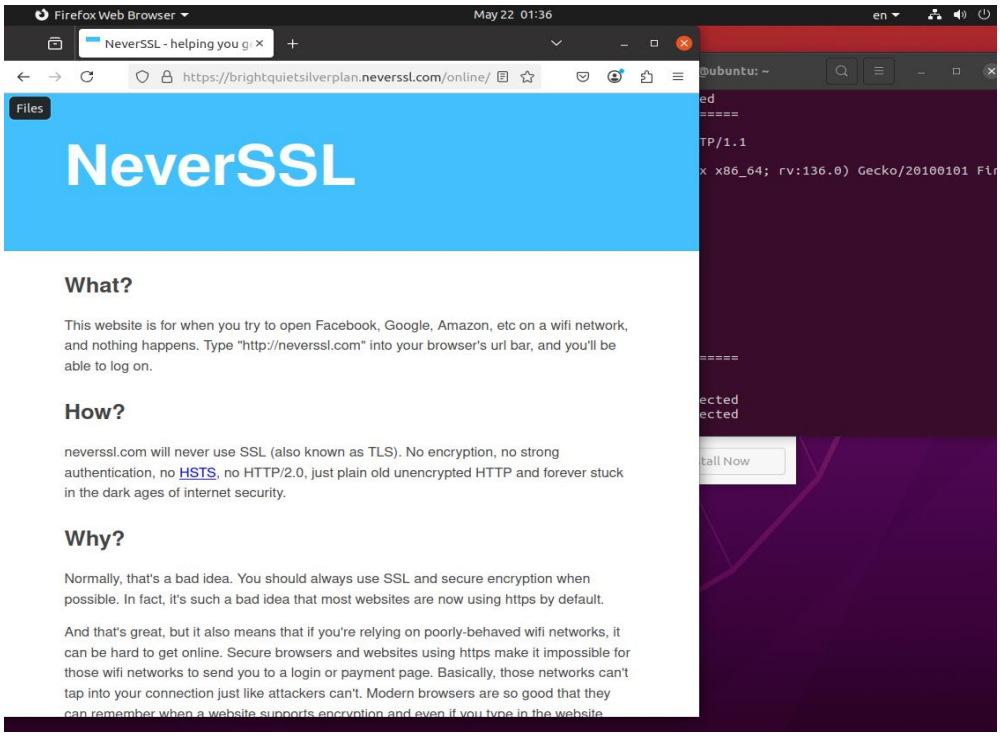


⇨ http://textfiles.com를 입력했을 때, 출력화면 , 프록시 서버로부터 받은 response message가 정상적으로 출력된 화면입니다.

⇨



⇨ http://www.catb.org/jargon/를 입력했을 때 출력화면 , 프록시 서버로부터 받은 response message가 정상적으로 출력된 화면입니다.



⇨ http://neverssl.com/ 를 입력했을 때, 프록시 서버로부터 받은 response message가 정상적으로 출력된 화면입니다.

```
kw2021202089@ubuntu:~$ cat ./logfile/logfile.txt
[Miss] ServerPID : 13117 | http://neverssl.com/ - [2025/5/22, 05:48:27]
[Miss] ServerPID : 13175 | http://www.catb.org/jargon/ - [2025/5/22, 05:48:32]
[Miss] ServerPID : 13206 | http://textfiles.com/ - [2025/5/22, 05:48:35]
[Miss] ServerPID : 13238 | http://httpbin.org/get - [2025/5/22, 05:48:39]
[Hit] ServerPID : 13264 | e64/a0a6639cc92cb8f1601e0260c54fb2b0c3a5e - [2025/5/22, 05:48:44]
[Hit]http://httpbin.org/get
[Miss] ServerPID : 13294 | http://silvermajesticastoundingmorning.neverssl.com/online - [2025/5/22, 05:48:56]
[Miss] ServerPID : 13295 | http://silvermajesticastoundingmorning.neverssl.com/online/ - [2025/5/22, 05:48:56]
**SERVER** [Terminated] run time: 41 sec. #sub process: 29
```

⇨ logfile.txt에 hit/miss 및 작동시간과 자식 프로세스 수에 대한 종료 문구가 작성된 것을 확인할 수 있습니다.

고찰

이번 과제는 이전에 구현한 proxy 2-3 을 바탕으로, SIGINT 신호에 대해서, Signal handler function을 활용하여서 "^c"가 입력되면 , 이 신호를 catch 하도록 합니다.

또한 입력받은 URL들의 HTTP request 에 대해서, 전부 hash function을 통해서, Cache 파일에 저장되도록 하도록 합니다.

이전에 작성된 Cache 파일을 바탕으로 입력받은 url이 cache file에 존재하면, HIT이 logfile.txt에 작성되도록하고, 존재하지 않으면, MISS 가 작성되도록 합니다.

그리고 SIGINT 신호가 catch 될 경우에는 프로그램이 종료되는데, 전체 작동시간 및 자식 프로세스의 수 또한 logfile.txt에 기록합니다.

이번 과제를 수행하면서, 궁금하였던 점이, logfile.txt를 보면, 마지막 줄에 자식 프로세스의 수를 확인할 수 있는데, 처음에 많이 생성된 자식 프로세스의 수를 보고, 왜 이렇게 많은 자식 프로세스가 생성되었는지 궁금하였습니다.

그래서 이를 디버그 메시지를 통해서 검증해보니까, 브라우저가 한번의 접속에 여러 개의 요청을 보내는데, 예를 들면 HTML, CSS, 이미지 등을 보낸다고 해보면, 그러면 이 각각이 하나의 개별 HTTP 요청이 되고, 이것에 대해서 Proxy 서버는 각 요청마다 fork()를 하여, 자식프로세스를 만들어서 처리합니다. 이로 인해서 자식 프로세스의 수가 많이 나오는 것이었고, 즉, 브라우저가 비동기적으로 병렬 요청을 보낸다는 것임을 알 수 있었습니다.

이번 과제를 통해 , 웹 브라우저와 Proxy 서버간의 연결 및 요청을 다룰 수 있었는데, 확실히 이번 과제에서 여러 테스트 사이트를 이용하여, 검증을 해보니까 HIT일 때와 MISS 일 때의 연결 속도가 HIT일 때 확실히 빠름을 체감할 수 있었습니다.

Reference